

Experiment 'p_6_1_b_process_array_every_even_element_seq'

Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing assume : 0
Responses containing expect : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_1_b_process_array_every_even_element_seq

Natural language statement: null

Method signature: p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (even_elements: seq<int>)

Ensures

- even_elements == evens(inputs)

Requires

- |inputs| == 10

Functional Code Given

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (even_elements:
→ seq<int>)

The method should respect the following contract:

requires |inputs| == 10, ensures even_elements == evens(inputs)

The contract uses the following dafny code:

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNTjAVbFLKDIU2uRXruxSjVEknF4

Model: gpt-5.5-2026-04-23

Created at: 1780660695

Finish reason: stop

Usage: 438 tokens in, and 5438 tokens out

```
{
  assert inputs[0..] == inputs;
  assert inputs[10..] == [];

  assert |inputs[0..]| == 10;
  assert (inputs[0..])[0] == inputs[0];
  assert (inputs[0..])[1..] == inputs[1..];

  assert |inputs[1..]| == 9;
```

```

assert (inputs[1..])[0] == inputs[1];
assert (inputs[1..])[1..] == inputs[2..];

assert |inputs[2..| == 8;
assert (inputs[2..])[0] == inputs[2];
assert (inputs[2..])[1..] == inputs[3..];

assert |inputs[3..| == 7;
assert (inputs[3..])[0] == inputs[3];
assert (inputs[3..])[1..] == inputs[4..];

assert |inputs[4..| == 6;
assert (inputs[4..])[0] == inputs[4];
assert (inputs[4..])[1..] == inputs[5..];

assert |inputs[5..| == 5;
assert (inputs[5..])[0] == inputs[5];
assert (inputs[5..])[1..] == inputs[6..];

assert |inputs[6..| == 4;
assert (inputs[6..])[0] == inputs[6];
assert (inputs[6..])[1..] == inputs[7..];

assert |inputs[7..| == 3;
assert (inputs[7..])[0] == inputs[7];
assert (inputs[7..])[1..] == inputs[8..];

assert |inputs[8..| == 2;
assert (inputs[8..])[0] == inputs[8];
assert (inputs[8..])[1..] == inputs[9..];

assert |inputs[9..| == 1;
assert (inputs[9..])[0] == inputs[9];
assert (inputs[9..])[1..] == inputs[10..];

even_elements :=
  if inputs[0] % 2 == 0 then [inputs[0]] +
    (if inputs[1] % 2 == 0 then [inputs[1]] +
      (if inputs[2] % 2 == 0 then [inputs[2]] +
        (if inputs[3] % 2 == 0 then [inputs[3]] +
          (if inputs[4] % 2 == 0 then [inputs[4]] +
            (if inputs[5] % 2 == 0 then [inputs[5]] +
              (if inputs[6] % 2 == 0 then [inputs[6]] +
                (if inputs[7] % 2 == 0 then [inputs[7]] +
                  (if inputs[8] % 2 == 0 then [inputs[8]] +
                    (if inputs[9] % 2 == 0 then [inputs[9]] else [])
                  else
                    (if inputs[9] % 2 == 0 then [inputs[9]] else [])))
                else
                (if inputs[9] % 2 == 0 then [inputs[9]] else [])))
            else
            (if inputs[9] % 2 == 0 then [inputs[9]] else [])))
          else
          (if inputs[9] % 2 == 0 then [inputs[9]] else [])))
        else
        (if inputs[9] % 2 == 0 then [inputs[9]] else [])))
      else
      (if inputs[8] % 2 == 0 then [inputs[8]] +
        (if inputs[9] % 2 == 0 then [inputs[9]] else []))
    )
  )

```


Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(84,74): Error: else
  ↪ expected
  |
84 | (if inputs[9] % 2 == 0 then [inputs[9]] else []))
  | ^

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(108,75): Error:
  ↪ else expected
  |
108 | (if inputs[9] % 2 == 0 then [inputs[9]] else []))
  | ^

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(145,74): Error:
  ↪ else expected
  |
145 | (if inputs[9] % 2 == 0 then [inputs[9]] else []))
  | ^

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(169,75): Error:
  ↪ else expected
  |
169 | (if inputs[9] % 2 == 0 then [inputs[9]] else []))));
  | ^

problem_data/p_6_1_b_process_array_every_even_element_seq/program.dfy(169,77): Error:
  ↪ closeparen expected
  |
169 | (if inputs[9] % 2 == 0 then [inputs[9]] else []))));
  | ^

5 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNVOElCu8NfvqqQ0AdtNex0sRSR

Model: gpt-5.5-2026-04-23

Created at: 1780660798

Finish reason: stop

Usage: 3427 tokens in, and 4410 tokens out

```
{
  var r10: seq<int> := [];
  assert inputs[10..] == [];
```

```

assert r10 == evens(inputs[10..]);

var r9: seq<int> := if inputs[9] % 2 == 0 then [inputs[9]] + r10 else r10;
assert |inputs[9..]| == 1;
assert (inputs[9..])[0] == inputs[9];
assert (inputs[9..])[1..] == inputs[10..];
assert r9 == evens(inputs[9..]);

var r8: seq<int> := if inputs[8] % 2 == 0 then [inputs[8]] + r9 else r9;
assert |inputs[8..]| == 2;
assert (inputs[8..])[0] == inputs[8];
assert (inputs[8..])[1..] == inputs[9..];
assert r8 == evens(inputs[8..]);

var r7: seq<int> := if inputs[7] % 2 == 0 then [inputs[7]] + r8 else r8;
assert |inputs[7..]| == 3;
assert (inputs[7..])[0] == inputs[7];
assert (inputs[7..])[1..] == inputs[8..];
assert r7 == evens(inputs[7..]);

var r6: seq<int> := if inputs[6] % 2 == 0 then [inputs[6]] + r7 else r7;
assert |inputs[6..]| == 4;
assert (inputs[6..])[0] == inputs[6];
assert (inputs[6..])[1..] == inputs[7..];
assert r6 == evens(inputs[6..]);

var r5: seq<int> := if inputs[5] % 2 == 0 then [inputs[5]] + r6 else r6;
assert |inputs[5..]| == 5;
assert (inputs[5..])[0] == inputs[5];
assert (inputs[5..])[1..] == inputs[6..];
assert r5 == evens(inputs[5..]);

var r4: seq<int> := if inputs[4] % 2 == 0 then [inputs[4]] + r5 else r5;
assert |inputs[4..]| == 6;
assert (inputs[4..])[0] == inputs[4];
assert (inputs[4..])[1..] == inputs[5..];
assert r4 == evens(inputs[4..]);

var r3: seq<int> := if inputs[3] % 2 == 0 then [inputs[3]] + r4 else r4;
assert |inputs[3..]| == 7;
assert (inputs[3..])[0] == inputs[3];
assert (inputs[3..])[1..] == inputs[4..];
assert r3 == evens(inputs[3..]);

var r2: seq<int> := if inputs[2] % 2 == 0 then [inputs[2]] + r3 else r3;
assert |inputs[2..]| == 8;
assert (inputs[2..])[0] == inputs[2];
assert (inputs[2..])[1..] == inputs[3..];
assert r2 == evens(inputs[2..]);

var r1: seq<int> := if inputs[1] % 2 == 0 then [inputs[1]] + r2 else r2;

```

```

assert |inputs[1..]| == 9;
assert (inputs[1..])[0] == inputs[1];
assert (inputs[1..])[1..] == inputs[2..];
assert r1 == evens(inputs[1..]);

var r0: seq<int> := if inputs[0] % 2 == 0 then [inputs[0]] + r1 else r1;
assert |inputs[0..]| == 10;
assert inputs[0..] == inputs;
assert (inputs[0..])[0] == inputs[0];
assert (inputs[0..])[1..] == inputs[1..];
assert r0 == evens(inputs[0..]);

even_elements := r0;
assert even_elements == evens(inputs);
}

```

Final Program

```

function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}

method p_6_1_b_process_array_every_even_element_seq(inputs: seq<int>) returns (
  ↪ even_elements: seq<int>)
  requires |inputs| == 10
  ensures even_elements == evens(inputs)
{
  var r10: seq<int> := [];
  assert inputs[10..] == [];
  assert r10 == evens(inputs[10..]);

  var r9: seq<int> := if inputs[9] % 2 == 0 then [inputs[9]] + r10 else r10;
  assert |inputs[9..]| == 1;
  assert (inputs[9..])[0] == inputs[9];
  assert (inputs[9..])[1..] == inputs[10..];
  assert r9 == evens(inputs[9..]);

  var r8: seq<int> := if inputs[8] % 2 == 0 then [inputs[8]] + r9 else r9;
  assert |inputs[8..]| == 2;
  assert (inputs[8..])[0] == inputs[8];
  assert (inputs[8..])[1..] == inputs[9..];
  assert r8 == evens(inputs[8..]);

  var r7: seq<int> := if inputs[7] % 2 == 0 then [inputs[7]] + r8 else r8;
  assert |inputs[7..]| == 3;
  assert (inputs[7..])[0] == inputs[7];
  assert (inputs[7..])[1..] == inputs[8..];
}

```

```

assert r7 == evens(inputs[7..]);

var r6: seq<int> := if inputs[6] % 2 == 0 then [inputs[6]] + r7 else r7;
assert |inputs[6..]| == 4;
assert (inputs[6..])[0] == inputs[6];
assert (inputs[6..])[1..] == inputs[7..];
assert r6 == evens(inputs[6..]);

var r5: seq<int> := if inputs[5] % 2 == 0 then [inputs[5]] + r6 else r6;
assert |inputs[5..]| == 5;
assert (inputs[5..])[0] == inputs[5];
assert (inputs[5..])[1..] == inputs[6..];
assert r5 == evens(inputs[5..]);

var r4: seq<int> := if inputs[4] % 2 == 0 then [inputs[4]] + r5 else r5;
assert |inputs[4..]| == 6;
assert (inputs[4..])[0] == inputs[4];
assert (inputs[4..])[1..] == inputs[5..];
assert r4 == evens(inputs[4..]);

var r3: seq<int> := if inputs[3] % 2 == 0 then [inputs[3]] + r4 else r4;
assert |inputs[3..]| == 7;
assert (inputs[3..])[0] == inputs[3];
assert (inputs[3..])[1..] == inputs[4..];
assert r3 == evens(inputs[3..]);

var r2: seq<int> := if inputs[2] % 2 == 0 then [inputs[2]] + r3 else r3;
assert |inputs[2..]| == 8;
assert (inputs[2..])[0] == inputs[2];
assert (inputs[2..])[1..] == inputs[3..];
assert r2 == evens(inputs[2..]);

var r1: seq<int> := if inputs[1] % 2 == 0 then [inputs[1]] + r2 else r2;
assert |inputs[1..]| == 9;
assert (inputs[1..])[0] == inputs[1];
assert (inputs[1..])[1..] == inputs[2..];
assert r1 == evens(inputs[1..]);

var r0: seq<int> := if inputs[0] % 2 == 0 then [inputs[0]] + r1 else r1;
assert |inputs[0..]| == 10;
assert inputs[0..] == inputs;
assert (inputs[0..])[0] == inputs[0];
assert (inputs[0..])[1..] == inputs[1..];
assert r0 == evens(inputs[0..]);

even_elements := r0;
assert even_elements == evens(inputs);
}

```

Total Token Usage

Input tokens: 3865

Output tokens: 9848

Reasoning tokens: 6267

Sum of 'total tokens': 13713

Experiment Timings

Overall Experiment started at 1780660693953, ended at 1780660909995, lasting 216042ms (216.04 seconds)

Iteration #1 started at 1780660693971, ended at 1780660797072, lasting 103101ms (103.10 seconds)

Iteration #2 started at 1780660797097, ended at 1780660909995, lasting 112898ms (112.90 seconds)